

گزارش تمرین ۳

مهسا ابرغانی اقدام – شماره دانشجویی : ۸۱۰۱۰۳۰۵۳

الف: اجرا با استفاده از blas:

محاسبه‌ی ضرب دو ماتریس مربعی را در مورد الف انجام می‌دهیم که یک بار اندازه‌ی ماتریس 10000×10000 و بار دیگر 20000×20000 است. یک ماتریس رندوم ایجاد کرده و در هم با استفاده از نامپای ضرب کرده و سپس خروجی را نمایش می‌دهیم. حالت پیش‌فرض blas می‌باشد.

:۱۰۰۰۰

```
1 import numpy as np
2 import time
3
4 def benchmark_matrix_multiplication(size):
5     A = np.random.rand(size, size)
6     B = np.random.rand(size, size)
7
8     start_time = time.time()
9     C = np.dot(A, B)
10    end_time = time.time()
11
12    print(f"Matrix multiplication of size {size}x{size} with BLAS took {end_time - start_time:.2f} seconds.")
13
14    benchmark_matrix_multiplication(10000)
```

:۲۰۰۰۰

```
1 import numpy as np
2 import time
3
4
5 def benchmark_matrix_multiplication(size):
6     A = np.random.rand(size, size)
7     B = np.random.rand(size, size)
8     start_time = time.time()
9     C = np.dot(A, B)
10    end_time = time.time()
11
12    print(f"Matrix multiplication of size {size}x{size} with BLAS took {end_time - start_time:.2f} seconds.")
13
14    benchmark_matrix_multiplication(20000)
```

نتیجه:

```
PS B:\Master\Distirbuted ml\3> & "C:/Program Files/Python310/python.exe" "b:/Master/Distirbuted ml/3/BLAS/Bench_a_10k.py"
Matrix multiplication of size 10000x10000 with BLAS took 6.86 seconds.
PS B:\Master\Distirbuted ml\3> & "C:/Program Files/Python310/python.exe" "b:/Master/Distirbuted ml/3/BLAS/Bench_a_20k.py"
Matrix multiplication of size 20000x20000 with BLAS took 94.19 seconds.
PS B:\Master\Distirbuted ml\3> 
```

الف: اجرا با استفاده از MKL

این بار همان ماتریس ها را با MKL اجرا می کنیم. MKL را روی ترمینال آنکوندا و با استفاده از دستور زیر نصب می کنیم:

```
"conda create -n np_mkl -c anaconda python=3.9 numpy mkl="2019.*" blas="*"*mkl
```

و با استفاده از دستور زیر فعال می کنیم:

```
conda activate np_mkl
```

:۱۰۰۰۰

```
1 import numpy as np
2 import time
3
4 def benchmark_matrix_multiplication(size):
5     A = np.random.rand(size, size)
6     B = np.random.rand(size, size)
7
8     start_time = time.time()
9     C = np.dot(A, B)
10    end_time = time.time()
11
12    print(f"Matrix multiplication of size {size}x{size} with MKL took {end_time - start_time:.2f} seconds.")
13
14    benchmark_matrix_multiplication(10000)
```

:۲۰۰۰۰

```
1 import numpy as np
2 import time
3
4
5 def benchmark_matrix_multiplication(size):
6     A = np.random.rand(size, size)
7     B = np.random.rand(size, size)
8     start_time = time.time()
9     C = np.dot(A, B)
10    end_time = time.time()
11
12    print(f"Matrix multiplication of size {size}x{size} with MKL took {end_time - start_time:.2f} seconds.")
13
14    benchmark_matrix_multiplication(20000)
```

نتیجه:

```
PS B:\Master\Distirbuted ml\3> & C:/ProgramData/anaconda3/python.exe "b:/Master/Distirbuted ml/3/MKL/Bench_a_10k.py"
Matrix multiplication of size 10000x10000 with MKL took 6.84 seconds.
PS B:\Master\Distirbuted ml\3> & C:/ProgramData/anaconda3/python.exe "b:/Master/Distirbuted ml/3/MKL/Bench_a_20k.py"
Matrix multiplication of size 20000x20000 with MKL took 53.58 seconds.
PS B:\Master\Distirbuted ml\3>
```

در این تمرین سرعت mkl بهتر از blas می‌باشد و زمان کمتری طول می‌کشد تا برنامه را اجرا کند.

ب: اجرا با استفاده از blas:

در این قسمت یک ماتریس رندوم ایجاد کرده و سپس با استفاده از توابع داخلی نامپای معکوس این ماتریس را به دست آورده و یک بار با blas و بار دیگر با استفاده از mkl اجرا می‌کنیم:

:۱۰۰۰۰

```
1  import numpy as np
2  import time
3
4  A = np.random.rand(10000, 10000)
5
6  start_time = time.time()
7  A_inv = np.linalg.inv(A)
8  end_time = time.time()
9
10 print(f"Matrix inversion of size 20000x20000 with BLAS took {end_time - start_time:.2f} seconds.")
```

:۲۰۰۰۰

```
1  import numpy as np
2  import time
3
4  A = np.random.rand(20000, 20000)
5
6  start_time = time.time()
7  A_inv = np.linalg.inv(A)
8  end_time = time.time()
9
10 print(f"Matrix inversion of size 20000x20000 with BLAS took {end_time - start_time:.2f} seconds.")
```

نتیجه:

```
PS B:\Master\Distirbuted ml\3> & "C:/Program Files/Python310/python.exe" "b:/Master/Distirbuted ml/3/BLAS/Bench_b_10k.py"
Matrix inversion of size 10000x10000 with BLAS took 11.61 seconds.
PS B:\Master\Distirbuted ml\3> & "C:/Program Files/Python310/python.exe" "b:/Master/Distirbuted ml/3/BLAS/Bench_b_20k.py"
Matrix inversion of size 20000x20000 with BLAS took 84.93 seconds.
PS B:\Master\Distirbuted ml\3>
```

ب: اجرا با استفاده از MKL

:۱۰۰۰۰

```

1 import numpy as np
2 import time
3
4 A = np.random.rand(10000, 10000)
5
6 start_time = time.time()
7 A_inv = np.linalg.inv(A)
8 end_time = time.time()
9
10 print(f"Matrix inversion of size 10000x10000 with MKL took {end_time - start_time:.2f} seconds.")

```

:۲۰۰۰۰

```

1 ∨ import numpy as np
2 import time
3
4 A = np.random.rand(20000, 20000)
5
6 start_time = time.time()
7 A_inv = np.linalg.inv(A)
8 end_time = time.time()
9
10 print(f"Matrix inversion of size 20000x20000 with MKL took {end_time - start_time:.2f} seconds.")

```

نتیجه:

```

PS B:\Master\Distirbuted ml\3> & C:/ProgramData/anaconda3/python.exe "b:/Master/Distirbuted ml/3/MKL/Bench_b_10k.py"
Matrix inversion of size 10000x10000 with MKL took 11.38 seconds.
PS B:\Master\Distirbuted ml\3> & C:/ProgramData/anaconda3/python.exe "b:/Master/Distirbuted ml/3/MKL/Bench_b_20k.py"
Matrix inversion of size 20000x20000 with MKL took 98.89 seconds.
PS B:\Master\Distirbuted ml\3>

```

در این تمرین mkl عملکرد بهتری از blas در ماتریس با ابعاد 10000×10000 دارد و در ماتریس 20000×20000 blas عملکرد بهتری از mkl دارد.